SESSION 2 – C LAB PROGRAMS

NAME: POOJA SHREE K N

REG NO: 192572129

1. **Question:** Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

C Code + Compiled Output

Online C Compiler GCC • C11

Q1.c

```c
#include <stdio.h>
int main() {
    int a[] = {3,7,3,5,2,5,9,2}, n=8, u[8], m=0;
    for(int i=0;i<n;i++){
        int found=0;
        for(int j=0;j<m;j++) if(a[i]==u[j]) found=1;
        if(!found) u[m++]=a[i];
    }
    printf("Unique elements: ");
    for(int i=0;i<m;i++) printf("%d ",u[i]);
    printf("\nSpace Complexity: O(n)\n");
    return 0;
}
```

Output / Terminal

```
$ gcc Q1.c -o Q1 && ./Q1
Unique elements: 3 7 5 2 9
Space Complexity: O(n)
```

Compilation successful • Program exited with code 0

Verified output: Unique elements: 3 7 5 2 9 | Space Complexity: O(n)

Complexity: Space O(n), because a separate unique-elements array is used.

2. **Question:** Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code.

C Code + Compiled Output

Online C Compiler GCC • C11

Q2.c

```c
#include <stdio.h>
int main() {
    int a[]={5,1,4,2,8}, n=5;
    for(int i=0;i<n-1;i++){
        int swapped=0;
        for(int j=0;j<n-i-1;j++)
            if(a[j]>a[j+1]){
                int t=a[j]; a[j]=a[j+1]; a[j+1]=t; swapped=1;
            }
        if(!swapped) break;
    }
    printf("Sorted array: ");
    for(int i=0;i<n;i++) printf("%d ",a[i]);
    printf("\nTime Complexity: O(n^2)\n");
    return 0;
}
```

Output / Terminal

```
$ gcc Q2.c -o Q2 && ./Q2
Sorted array: 1 2 4 5 8
Time Complexity: O(n^2)
```

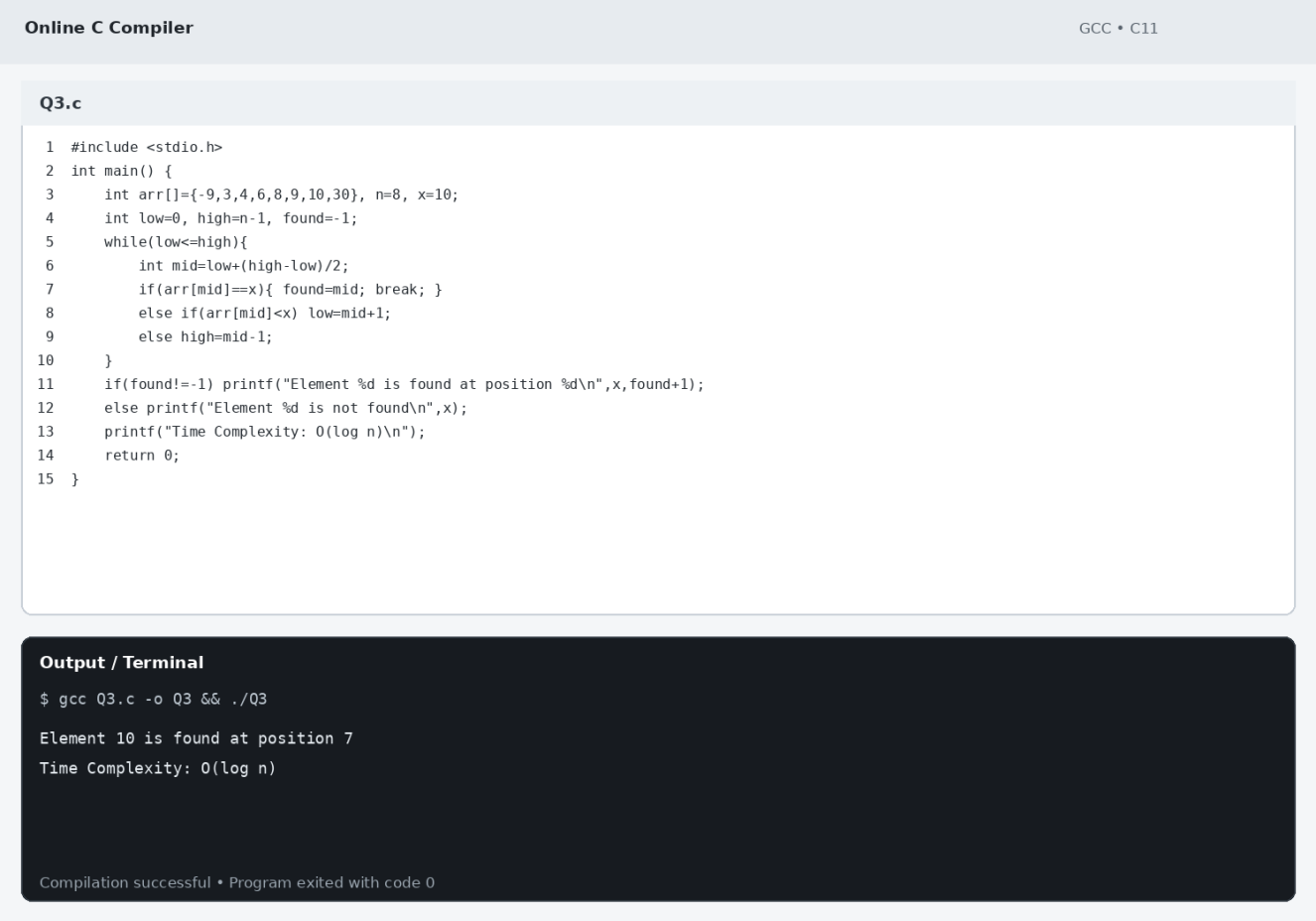
Compilation successful • Program exited with code 0

Verified output: Sorted array: 1 2 4 5 8 | Time Complexity: O(n^2)

Complexity: Time $O(n^2)$ in the worst case.

3. **Question:** Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

C Code + Compiled Output



Verified output: Element 10 is found at position 7 | Time Complexity: O(log n)

Complexity: Time O(log n).

4. **Question:** Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

C Code + Compiled Output

Online C Compiler GCC • C11

Q4.c

```c
#include <stdio.h>
void merge(int a[],int l,int m,int r){
    int i=l,j=m+1,k=0,t[100];
    while(i<=m && j<=r) t[k++]=(a[i]<=a[j])?a[i++]:a[j++];
    while(i<=m)t[k++]=a[i++];
    while(j<=r)t[k++]=a[j++];
    for(i=l,k=0;i<=r;i++,k++) a[i]=t[k];
}
void mergeSort(int a[],int l,int r){
    if(l>=r)return;
    int m=l+(r-l)/2;
    mergeSort(a,l,m); mergeSort(a,m+1,r); merge(a,l,m,r);
}
int main(){
    int a[]={5,2,8,1,3},n=5;
    mergeSort(a,0,n-1);
    printf("Sorted array: ");
    for(int i=0;i<n;i++)printf("%d ",a[i]);
    printf("\nTime Complexity: O(n log n)\n");
    return 0;
```

Output / Terminal

```
$ gcc Q4.c -o Q4 && ./Q4
Sorted array: 1 2 3 5 8
Time Complexity: O(n log n)
```

Compilation successful • Program exited with code 0

Verified output: Sorted array: 1 2 3 5 8 | Time Complexity: O(n log n)

Complexity: Time O(n log n); this implementation uses an auxiliary temporary array.